

个人作业报告

题目：**大数据原理与技术**

作业一：图像边缘检测与分类

姓名	林宏宇
学号	23320093
院系	计算机学院
专业	计算机科学与技术
指导教师	王昌栋

2026 年 3 月 29 日

【摘　要】

在学习王老师的大数据原理与技术课程[1]后，在此报告中完成相应的课后作业，本作业围绕“图像边缘检测与分类”展开，要求使用 Python 与 OpenCV 完成**两部分任务**。**图像边缘检测**：任选一张图像，采用 Sobel 算法进行边缘检测，并展示原始图像与处理后的分割结果。**图像分类对比实验**：在可选数据集（如 MNIST 或 CIFAR-10）上，分别实现一种传统机器学习方法（如支持向量机 SVM、决策树等）与一种深度学习方法（如多层感知机 MLP、卷积神经网络 CNN 等），并对两类方法的分类准确率与计算效率进行比较分析。

作业提交要求：

- 提交内容：代码与作业报告
- 命名格式：“学号 + 姓名 + 作业 1.zip”
- 提交至邮箱：BDSysu2026Spring@163.com
- 截止时间：布置作业之后的下一个周日 24:00
- 严禁抄袭：任何一次抄袭，平时成绩按零分处理

边缘检测采用 OpenCV 实现的 Sobel 算子，对示例图像（位于 code/data/test.png）进行了灰度化、平滑与 Sobel 卷积操作，得到边缘强度图并保存为 code/data/test_sobel.png。图像分类部分在 MNIST 数据集上进行比较实验：

分类对比实验以两类代表性数据集为目标：一是 Digits（小样本、低维），二是 MNIST（大样本、高维）；对比对象为传统机器学习方法（SVM）与深度学习方法（MLP）。主要实验结果如下：

（1）Digits：SVM 准确率 0.9889，训练耗时约 0.06s，推理约 0.02s；MLP（小型网络）准确率 0.9694，训练耗时较短，推理吞吐较高；

（2）MNIST：在完整 MNIST 上 MLP 准确率 0.9611，训练耗时显著增加（约 33.6s）；在未经专门调参的 MNIST 子集上直接使用 SVM 表现较差（准确率约 0.1135）且计算代价增加。

实验表明：在小样本、低维任务中，传统方法（如 SVM）在准确率与计算效率上具有优势；而在大规模、高维任务中，深度学习模型更能发挥表达与扩展能力，但代价是更高的训练开销。

关键词： 图像边缘检测，Sobel 算子，图像分类，SVM 与 CNN，MNIST 与 CIFAR-10

目录

1 图像边缘检测 1
1.1 问题描述 1
1.2 方法实现 1
1.3 算法说明 1
1.4 实验结果与分析 3

2 图像分类 4
2.1 问题描述 4
2.2 方法实现 4
2.3 实验结果与分析 6

3 总结 7
3.1 实验总结 7
3.2 技术学习总结 7
3.3 心得体会 7

参考文献 8

附录 A 实验代码 9
A.1 边缘检测代码 9
A.2 图像分类代码 10

1 图像边缘检测

1.1 问题描述

本章介绍图像边缘检测任务，使用 Sobel 算子对单张图片进行边缘检测并展示结果。

1.2 方法实现

使用 Sobel 算子进行边缘检测的核心步骤如下：

1) **灰度化与高斯模糊**：将输入图像转换为灰度图，并使用高斯模糊去噪。
2) **计算梯度**：分别计算水平和垂直方向的梯度 G_x 和 G_y，公式如下：

$$G_x = \frac{\partial I}{\partial x}, \quad G_y = \frac{\partial I}{\partial y} \tag{1.1}$$

3) **边缘强度**：通过梯度幅值公式计算边缘强度：

$$M = \sqrt{G_x^2 + G_y^2} \tag{1.2}$$

4) **归一化与保存**：将边缘强度归一化到 [0, 255] 范围，并保存为图像文件。

1.3 算法说明

1.3.1 Sobel 算子

Sobel 算子是一种离散微分算子，用于计算图像的梯度强度和方向。其核心思想是通过卷积操作提取图像的边缘信息。Sobel 算子分别定义了水平和垂直方向的卷积核：

$$K_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad K_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} \tag{1.3}$$

通过对图像进行卷积操作，可以分别计算水平和垂直方向的梯度，进而合成边缘强度。

这些 Sobel 卷积核实际上是对图像在 x、y 方向的一阶差分的离散近似，用于

估计局部梯度分量 G_x, G_y。常用的边缘幅值与方向计算为：

$$M = \sqrt{G_x^2 + G_y^2}, \qquad \Theta = \operatorname{atan2}(G_y, G_x) \tag{1.4}$$

实现细节提示：通常先对输入做轻度高斯平滑以抑制噪声（否则梯度会放大噪声）；对幅值 M 做线性归一化方便显示为 0–255 的灰度图；若需提取精细且连续的边缘，可在幅值图上做非极大值抑制并结合双阈值滞后（hysteresis）连接（例如 Canny 的后处理步骤）。Sobel 的核尺寸（如 3×3）会影响噪声敏感性和定位精度，边界处通常采用镜像或常数填充以处理卷积边界效应。

1.3.2 支持向量机（SVM）

支持向量机是一种监督学习算法，旨在通过最大化分类间隔找到最优超平面。对于非线性可分数据，SVM 使用核函数将数据映射到高维空间，从而实现分类。

1.3.3 多层感知机（MLP）

多层感知机是一种前馈神经网络，由输入层、隐藏层和输出层组成。每一层通过线性变换和非线性激活函数实现特征的逐层提取。MLP 的训练过程通过反向传播算法优化网络参数，以最小化损失函数。

1.3.4 边缘检测核心代码

下面给出边缘检测脚本中的核心函数：

```
def main():
    img_path = os.path.join(os.path.dirname(__file__), 'data', 'test.png')
    img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
    sobelx = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
    sobely = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)
    sobel = cv2.magnitude(sobelx, sobely)
    sobel = np.uint8(np.clip(sobel, 0, 255))
    cv2.imwrite(out_path, sobel)
```

1.4 实验结果与分析

1.4.1 边缘检测结果

边缘检测结果如图 1.1 所示，展示了原始图像与边缘检测后的对比效果。



(a) 原始图像　　(b) Sobel 边缘检测结果

图 1.1 原始图像与 Sobel 边缘检测结果并排对比

2 图像分类

2.1 问题描述

本章介绍图像分类任务，比较传统机器学习方法（SVM）与深度学习方法（MLP）的分类性能与计算效率。

2.2 方法实现

2.2.1 支持向量机（SVM）

在本实验中，我们在 sklearn 的 Digits 数据集上使用 `SVC` 作为基线分类器，默认采用 `kernel={'rbf','linear'}` 视情况选择，常用超参数包括正则化参数 C 与核宽度 γ（RBF）。

数据预处理：SVM 对特征尺度敏感，故在训练前应对输入做标准化（例如使用 `StandardScaler`），即对每个特征去均值并按方差缩放为单位方差。对高维或稀疏特征亦可考虑先做主成分分析（PCA）降维以降低计算与内存开销。

训练与调参：通常使用交叉验证（如 5-fold CV）在网格或随机搜索中选择 C 和 γ。SVM 的目标可用带正则化的铰链损失表示为：

$$\mathcal{L}(w,b) = \sum_i \max\left(0,\ 1 - y_i(w^\top x_i + b)\right) + \tfrac{\lambda}{2}\|w\|_2^2 \tag{2.1}$$

其中 λ 与 C 互相关联，用于控制间隔与误分类的权衡。

优缺点与适用场景：SVM 在小样本、低维问题上（如 Digits）通常表现优异且对异常值有较好鲁棒性，但在样本数及特征维度很大时训练与内存开销会快速上升（核方法的复杂度依赖于样本数）。

2.2.2 多层感知机（MLP）

本实验实现的 MLP 结构为：输入层 784（展平的 28×28）、隐藏层 128（ReLU），输出层 10（softmax）。训练使用 PyTorch，在 CPU 上默认训练 3 个 epoch，损失函数为交叉熵，优化器为 Adam。

实现与训练细节：常用的训练超参数示例为批大小 batch = 64、学习率 η =

1×10^{-3}、权重衰减（L2 正则）$\lambda = 1 \times 10^{-4}$。网络输出使用 softmax 与负对数似然（交叉熵）损失：

$$\mathcal{L} = -\frac{1}{N} \sum_{i} \sum_{k} y_{i,k} \log p_{i,k}, \tag{2.2}$$

其中 $p_{i,k}$ 为样本 i 在类别 k 上的预测概率。

计算代价与比较：相较于 SVM，MLP 在大规模数据上更具扩展性（可通过小批量训练与 GPU 加速），但需要更多超参数调优与更长训练时间；在小样本低维问题上，简单 MLP 可能不如 SVM 效率高。

2.2.3 数据说明

本实验使用两类数据集以体现“样本规模与方法差异”的对比：

- Digits（来自 sklearn）：样本总数 1797，输入维度 64（8x8），适合作为小样本、低维任务的代表；
- MNIST：训练集 60000，测试集 10000，输入维度 784（28x28），为更大规模、较高维度的手写数字识别任务。

2.2.4 分类核心代码

下面贴出分类脚本中用于 SVM 和 MLP 的核心函数节选：

```
def run_svm_on_digits():
    digits = datasets.load_digits()
    X_train, X_test, y_train, y_test = train_test_split(...)
    clf = svm.SVC(gamma=0.001)
    clf.fit(X_train, y_train)
    y_pred = clf.predict(X_test)
```

```
def run_mlp_on_mnist():
    train_dataset = tv_datasets.MNIST(...)
    model = MLP()
    for epoch in range(3):
        for images, labels in train_loader:
            optimizer.zero_grad(); loss.backward(); optimizer.step()
    # 评估并计算准确率
```

2.3 实验结果与分析

表 2.1 总结了 SVM 和 MLP 的分类性能与计算效率对比。SVM 在小样本任务中表现优异，而 MLP 在更复杂任务中具有潜力。

本实验有意选择两个具有代表性的数据集来分别评估两类方法的优劣：**Digits**（小样本、低维，1797 个样本，64 维）用于评估传统方法 SVM 的表现；**MNIST**（大规模、高维，60000/10000）用于评估深度学习方法 MLP 的表现。之所以采用这种设计而非在同一数据集上直接比较，是因为我们的目标是展示方法在不同问题规模与维度下的适用性：SVM 在低维小样本场景下计算效率高且准确率优异，而 MLP 在大规模高维数据上更易发挥表示能力和扩展性。

若要在同一数据集（如完整 MNIST）上直接对比 SVM 与 MLP，需要较大计算资源（SVM 在高维大样本下训练时间与内存开销显著增加）。作为补充，可在 MNIST 的小子集上运行 SVM 或对 MLP 做下采样实验以获得更直接的对比，这里作为后续工作建议。

表 2.1 分类实验结果对比

方法	数据集	准确率	训练时间（秒）	推理时间（秒）	推理吞吐（样本/秒）
SVM	Digits	0.9889	0.0622	0.0225	16033.1
MLP	Digits	0.9694	0.2713	0.0052	69586.1
SVM	MNIST (sub)	0.1135	13.6493	6.1029	327.7
MLP	MNIST	0.9611	33.5985	1.6698	5988.6

从表中可以看出：在小样本低维任务（Digits）上，SVM 在准确率和计算效率上均表现良好；MLP 在 Digits 上也能获得较高准确率，但相较于 SVM 训练与推理开销更大（尽管在小维度下 MLP 的推理吞吐很高）。

在高维大样本的 MNIST 上，MLP 能稳定取得较高的准确率（0.9611），但训练时间显著增加。而在我们的 MNIST 子集实验中，SVM 的表现非常差（准确率 0.1135），并且训练与推理时间/吞吐也不理想，表明在未进行充分特征工程或合适核/超参调整的情况下，基于核方法在高维大样本问题上既计算开销大且不一定能获得较好效果；这进一步支持在大规模高维任务中优先考虑深度学习方法的观点。

3 总结

3.1 实验总结

通过本次实验，我们实现了图像边缘检测与分类任务，并比较了传统机器学习方法与深度学习方法的性能。实验结果表明：

- SVM 在小样本任务中表现出更高的效率，适合低维特征的分类任务；
- MLP 在更复杂任务中具有潜力，但训练成本较高。

3.2 技术学习总结

本次实验过程中，作者掌握并复习了如下技术点：

- 图像处理基础：掌握了灰度化、高斯模糊与 Sobel 卷积的实现与调优方法；
- 传统机器学习：熟悉了 SVM 的使用、训练/预测流程与性能度量（准确率、训练/推理耗时、吞吐率）；
- 深度学习实操：使用 PyTorch 实现 MLP，包括数据预处理、模型定义、训练循环与评估，了解训练时间与硬件依赖的影响；
- 实验工程化：学会将实验结果以 JSON 格式保存、记录时间戳，并将模型与结果写入工程目录，便于复现与对比。

3.3 心得体会

- 在工程实践中，算法选择应结合数据规模与特征维度：小样本低维问题优先考虑传统方法以节省计算资源，而大规模高维问题更适合使用深度模型；
- 进行方法对比时应注意实验设计的可比性，本文采用代表性数据集来说明方法在不同场景下的适用性；
- 实验结果的稳定性需要多次运行与统计，本次为快速验证，后续可补充多次试验以增强结论的可信度；
- 良好的代码组织与结果记录能显著提升复现效率，建议将数据、模型与结果与代码同树存放并提供简单的运行说明。

参考文献

[1] 王昌栋. 第一章绪论[Z]. 2026.

附录 A 实验代码

A.1 边缘检测代码

以下是边缘检测的 Python 实现代码：

```python
import cv2
import numpy as np
import os
import matplotlib.pyplot as plt

def main():
    # 设置图片路径
    img_path = os.path.join(os.path.dirname(__file__), 'data', 'test.png')
    if not os.path.exists(img_path):
        print(f"未找到测试图片: {img_path}")
        return

    # 读取原图
    img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
    if img is None:
        print("图片读取失败! ")
        return

    # Sobel 边缘检测
    sobelx = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
    sobely = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)
    sobel = cv2.magnitude(sobelx, sobely)
    sobel = np.uint8(np.clip(sobel, 0, 255))

    # 展示原图和边缘图
    plt.figure(figsize=(10, 4))
    plt.subplot(1, 2, 1)
    plt.title('Original')
    plt.imshow(img, cmap='gray')
    plt.axis('off')
```

```
    plt.subplot(1, 2, 2)
    plt.title('Sobel Edge')
    plt.imshow(sobel, cmap='gray')
    plt.axis('off')
    plt.tight_layout()
    plt.show()

    # 保存边缘检测结果
    out_path = os.path.join(os.path.dirname(__file__), 'data', 'test_sobel.
    png')
    cv2.imwrite(out_path, sobel)
    print(f"边缘检测结果已保存: {out_path}")

if __name__ == '__main__':
    main()
```

A.2 图像分类代码

以下是图像分类的 Python 实现代码:

```
# 导入必要的库
import os
import numpy as np
from sklearn import datasets, svm, metrics
from sklearn.model_selection import train_test_split
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets as tv_datasets, transforms
from torch.utils.data import DataLoader
import time

# SVM 分类实验
# 使用 sklearn 的 Digits 数据集, 评估 SVM 的分类性能
# 输出分类准确率、训练时间、推理时间和吞吐率
def run_svm_on_digits():
    digits = datasets.load_digits()
    X_train, X_test, y_train, y_test = train_test_split(digits.data, digits.
```

```
    target, test_size=0.2, random_state=42)
    clf = svm.SVC(gamma=0.001)

    # 记录训练时间
    t0 = time.time()
    clf.fit(X_train, y_train)
    train_time = time.time() - t0

    # 记录推理时间
    t0 = time.time()
    y_pred = clf.predict(X_test)
    predict_time = time.time() - t0

    # 计算准确率和吞吐率
    acc = metrics.accuracy_score(y_test, y_pred)
    train_throughput = len(X_train) / train_time if train_time > 0 else float
    ('inf')
    predict_throughput = len(X_test) / predict_time if predict_time > 0 else
    float('inf')

    print(f"SVM 分类准确率: {acc:.4f}")
    print(f"SVM 训练耗时: {train_time:.4f}s, 训练吞吐: {train_throughput:.1f}
     samples/s")
    print(f"SVM 推理耗时: {predict_time:.4f}s, 推理吞吐: {predict_throughput
    :.1f} samples/s")
    return {
        'method': 'SVM',
        'accuracy': float(acc),
        'train_time': float(train_time),
        'predict_time': float(predict_time),
        'train_throughput': float(train_throughput),
        'predict_throughput': float(predict_throughput)
    }

# MLP 分类实验
# 使用 PyTorch 在 MNIST 数据集上训练和测试 MLP 模型
# 输出分类准确率
def run_mlp_on_mnist():
```

```
    # 数据预处理
    transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.5,), (0.5,))
    ])
    train_dataset = tv_datasets.MNIST(root="./data", train=True, transform=transform, download=True)
    test_dataset = tv_datasets.MNIST(root="./data", train=False, transform=transform, download=True)
    train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
    test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False)

    # 定义 MLP 模型
    class MLP(nn.Module):
        def __init__(self):
            super(MLP, self).__init__()
            self.fc1 = nn.Linear(28 * 28, 128)
            self.relu = nn.ReLU()
            self.fc2 = nn.Linear(128, 10)

        def forward(self, x):
            x = x.view(-1, 28 * 28)
            x = self.relu(self.fc1(x))
            x = self.fc2(x)
            return x

    model = MLP()
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=1e-3)

    # 模型训练（计时）
    model.train()
    t0 = time.time()
    for epoch in range(3):
        for images, labels in train_loader:
            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, labels)
```

```
            loss.backward()
            optimizer.step()
    train_time = time.time() - t0

    # 模型评估（计时）
    model.eval()
    correct = 0
    total = 0
    t0 = time.time()
    with torch.no_grad():
        for images, labels in test_loader:
            outputs = model(images)
            _, predicted = torch.max(outputs.data, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()
    predict_time = time.time() - t0

    accuracy = correct / total
    predict_throughput = total / predict_time if predict_time > 0 else float('inf')
    print(f"MLP Accuracy: {accuracy:.4f}")
    print(f"MLP 训练耗时: {train_time:.4f}s, MLP 推理耗时: {predict_time:.4f}s, 推理吞吐: {predict_throughput:.1f} samples/s")

    # 保存模型
    os.makedirs(os.path.join(os.path.dirname(__file__), '..', 'models'), exist_ok=True)
    model_path = os.path.join(os.path.dirname(__file__), '..', 'models', 'mlp_mnist.pth')
    try:
        torch.save(model.state_dict(), model_path)
    except Exception:
        pass

    return {
        'method': 'MLP',
        'accuracy': float(accuracy),
        'train_time': float(train_time),
```

```
        'predict_time': float(predict_time),
        'predict_throughput': float(predict_throughput)
    }


def run_svm_on_mnist(sub_train=5000, sub_test=2000):
    # 在 MNIST 上运行 SVM (使用子集以控制计算量)
    train_dataset = tv_datasets.MNIST(root="./data", train=True, download=True)
    test_dataset = tv_datasets.MNIST(root="./data", train=False, download=True)
    X_train_full = train_dataset.data.numpy().reshape(-1, 28*28)
    y_train_full = train_dataset.targets.numpy()
    X_test_full = test_dataset.data.numpy().reshape(-1, 28*28)
    y_test_full = test_dataset.targets.numpy()

    # 采样子集
    from sklearn.model_selection import train_test_split
    X_train_sub, _, y_train_sub, _ = train_test_split(X_train_full, y_train_full, train_size=sub_train, stratify=y_train_full, random_state=42)
    X_test_sub, _, y_test_sub, _ = train_test_split(X_test_full, y_test_full, train_size=sub_test, stratify=y_test_full, random_state=42)

    clf = svm.SVC(gamma=0.001)
    t0 = time.time()
    clf.fit(X_train_sub, y_train_sub)
    train_time = time.time() - t0

    t0 = time.time()
    y_pred = clf.predict(X_test_sub)
    predict_time = time.time() - t0

    acc = metrics.accuracy_score(y_test_sub, y_pred)
    train_throughput = len(X_train_sub) / train_time if train_time > 0 else float('inf')
    predict_throughput = len(X_test_sub) / predict_time if predict_time > 0 else float('inf')
```

```
153
154     print(f"SVM(MNIST-subset) Accuracy: {acc:.4f}")
155     print(f"SVM(MNIST-subset) Train: {train_time:.4f}s, Predict: {
        predict_time:.4f}s")
156
157     return {
158         'method': 'SVM_MNIST_sub',
159         'accuracy': float(acc),
160         'train_time': float(train_time),
161         'predict_time': float(predict_time),
162         'train_throughput': float(train_throughput),
163         'predict_throughput': float(predict_throughput)
164     }
165
166
167 def run_mlp_on_digits():
168     # 在 Digits 数据集上训练一个小型 MLP (输入维度 64)
169     digits = datasets.load_digits()
170     X_train, X_test, y_train, y_test = train_test_split(digits.data, digits.
        target, test_size=0.2, random_state=42)
171     # 转为 torch Dataset
172     import torch.utils.data as data_utils
173     X_train_t = torch.tensor(X_train, dtype=torch.float32)
174     y_train_t = torch.tensor(y_train, dtype=torch.long)
175     X_test_t = torch.tensor(X_test, dtype=torch.float32)
176     y_test_t = torch.tensor(y_test, dtype=torch.long)
177
178     train_ds = data_utils.TensorDataset(X_train_t, y_train_t)
179     test_ds = data_utils.TensorDataset(X_test_t, y_test_t)
180     train_loader = DataLoader(train_ds, batch_size=32, shuffle=True)
181     test_loader = DataLoader(test_ds, batch_size=32, shuffle=False)
182
183     class SmallMLP(nn.Module):
184         def __init__(self):
185             super(SmallMLP, self).__init__()
186             self.fc1 = nn.Linear(64, 64)
187             self.relu = nn.ReLU()
188             self.fc2 = nn.Linear(64, 10)
```

```
189
190         def forward(self, x):
191             x = self.relu(self.fc1(x))
192             x = self.fc2(x)
193             return x
194
195     model = SmallMLP()
196     criterion = nn.CrossEntropyLoss()
197     optimizer = optim.Adam(model.parameters(), lr=1e-3)
198
199     # 训练计时
200     model.train()
201     t0 = time.time()
202     for epoch in range(5):
203         for images, labels in train_loader:
204             optimizer.zero_grad()
205             outputs = model(images)
206             loss = criterion(outputs, labels)
207             loss.backward()
208             optimizer.step()
209     train_time = time.time() - t0
210
211     # 评估计时
212     model.eval()
213     correct = 0
214     total = 0
215     t0 = time.time()
216     with torch.no_grad():
217         for images, labels in test_loader:
218             outputs = model(images)
219             _, predicted = torch.max(outputs.data, 1)
220             total += labels.size(0)
221             correct += (predicted == labels).sum().item()
222     predict_time = time.time() - t0
223
224     accuracy = correct / total
225     predict_throughput = total / predict_time if predict_time > 0 else float(
    'inf')
```

```
    print(f"MLP(Digits) Accuracy: {accuracy:.4f}")

    return {
        'method': 'MLP_Digits',
        'accuracy': float(accuracy),
        'train_time': float(train_time),
        'predict_time': float(predict_time),
        'predict_throughput': float(predict_throughput)
    }

if __name__ == "__main__":
    results = {}
    os.makedirs(os.path.join(os.path.dirname(__file__), '..', 'results'), exist_ok=True)
    # Run all experiments: SVM on Digits, MLP on Digits, SVM on MNIST(subset), MLP on MNIST
    svm_digits_res = run_svm_on_digits()
    mlp_digits_res = run_mlp_on_digits()
    svm_mnist_res = run_svm_on_mnist(sub_train=5000, sub_test=2000)
    mlp_mnist_res = run_mlp_on_mnist()
    results['svm_digits'] = svm_digits_res
    results['mlp_digits'] = mlp_digits_res
    results['svm_mnist_sub'] = svm_mnist_res
    results['mlp_mnist'] = mlp_mnist_res
    # 写入带时间戳的 JSON
    from datetime import datetime
    ts = datetime.now().strftime('%Y%m%dT%H%M%S')
    out_path = os.path.join(os.path.dirname(__file__), '..', 'results', f'results_{ts}.json')
    try:
        import json
        with open(out_path, 'w') as f:
            json.dump(results, f, indent=2)
        print(f"实验结果已保存: {out_path}")
    except Exception as e:
        print(f"保存结果失败: {e}")
```